

Contexte

Plateforme bancaire sécurisée “**AI Baraka Digital**”

La banque **AI Baraka Digital** souhaite digitaliser la gestion de ses opérations bancaires pour ses clients et ses agents internes.

Les opérations incluent : **dépôts, retraits, virements**, et doivent être sécurisées, traçables, et conformes aux règles internes de validation pour des montants importants.

Problèmes identifiés

- Les opérations sensibles étaient traitées **manuellement**
- Risque de fraude ou **d'erreurs** sur les opérations importantes
- Difficulté de **traçabilité** et de contrôle interne
- Absence d'automatisation **sécurisée** pour les comptes clients et agents

Objectifs

- **Sécuriser** l'accès aux APIs via **JWT (stateless)**
- Gérer la logique métier pour **dépôts, retraits, virements**
- Implémenter des **workflows** de **validation** selon le montant
- Déployer l'application dans un **conteneur Docker**
- Préparer une base pour l'intégration future CI/CD

Rôles du système

Rôle	Actions principales
CLIENT	Créer compte, se connecter, créer opérations, upload justificatifs
AGENT_BANCAIRE	Consulter opérations PENDING, valider/rejeter opérations
ADMIN	Créer et gérer comptes clients et admins, gérer statut des comptes

Scénarios métier détaillés

1- Création de compte (Client)

- **Prérequis** : Le client n'a pas de compte.
- **Action** : Remplir le formulaire (email, mot de passe, nom complet)
- **Résultat attendu** :
 - Compte client créé
 - Numéro de compte unique généré automatiquement
 - Client peut maintenant se connecter

2- Login client

- **Prérequis** : Le client a un compte existant
- **Action** : Saisie email + mot de passe
- **Résultat attendu** :
 - Authentification réussie
 - JWT généré pour accéder aux opérations

3- Dépôt (Deposit)

- **Prérequis** : Client connecté, compte actif

- **Action** : Créer opération dépôt montant X
- **Cas A** : Montant $\leq$ 10 000 DH
 - Opération validée automatiquement
 - Solde du compte augmenté de X
- **Cas B** : Montant $>$ 10 000 DH
 - Upload justificatif (PDF/JPG/PNG max 5MB)
 - Opération passe en **PENDING**
 - Agent peut approuver ou rejeter
 - Si approuvée → solde augmenté
 - Si rejetée → solde inchangé

4- Retrait (Withdrawal)

- **Prérequis** : Client connecté, solde suffisant
- **Action** : Créer opération retrait montant X
- **Cas A** : Montant $\leq$ 10 000 DH
 - Opération validée automatiquement
 - Solde diminué de X
- **Cas B** : Montant $>$ 10 000 DH
 - Upload justificatif
 - Opération PENDING
 - Agent peut approuver ou rejeter
 - Solde mis à jour uniquement si approuvée

5- Virement (Transfer)

- **Prérequis** : Client connecté, solde du compte source suffisant
- **Action** : Créer opération transfert vers un autre compte montant X
- **Cas A** : Montant $\leq 10\,000$ DH
 - Opération validée automatiquement
 - Solde source diminué, solde destination augmenté
- **Cas B** : Montant $> 10\,000$ DH
 - Upload justificatif
 - Opération PENDING
 - Agent valide ou rejette
 - Solde modifié seulement si approuvée

6- Gestion par l'agent bancaire

- **Prérequis** : Agent connecté
- **Action** :
 - Consulter opérations PENDING
 - Vérifier documents uploadés
 - Approuver ou rejeter opérations
- **Résultat attendu** :
 - Approuver → solde mis à jour selon type
 - Rejeter → solde inchangé

7- Création / gestion comptes par admin

- **Prérequis** : Admin connecté
- **Action** :
 - Créer / gérer comptes clients ou autres admins

- **Résultat attendu :**

- Nouveau client/admin ajouté avec statut actif
- Accès au système selon rôle

Sécurité

- Authentification **JWT stateless**
- Spring Security 6 + UserDetailsService personnalisé
- PasswordEncoder : BCrypt
- Sécurisation des endpoints selon rôle :
 - CLIENT : `/api/client/**`
 - AGENT_BANCAIRE : `/api/agent/**`
 - ADMIN : `/api/admin/**`

Endpoint	Méthode	Rôle	Description
<code>/auth/login</code>	POST	Tous	Authentification + JWT
<code>/api/client/operations</code>	POST	CLIENT	Créer opération
<code>/api/client/operations/{id}/document</code>	POST	CLIENT	Upload justificatif
<code>/api/client/operations</code>	GET	CLIENT	Lister opérations
<code>/api/agent/operations/pending</code>	GET	AGENT	Lister opérations PENDING
<code>/api/agent/operations/{id}/approve</code>	PUT	AGENT	Approuver opération
<code>/api/agent/operations/{id}/reject</code>	PUT	AGENT	Rejeter opération
<code>/api/admin/users</code>	POST/PUT/DELETE	ADMIN	Gérer comptes

Modèle de données

User

- id, email, password, fullName, role, active, createdAt

Account

- id, accountNumber, balance, owner

Operation

- id, type, amount, status, createdAt, validatedAt, executedAt, accountSource, accountDestination

Document

- id, fileName, fileType, storagePath, uploadedAt, operation

Docker

- Dockerfile pour l'application backend
- Variables d'environnement :
 - `JWT_SECRET`
 - `DB_URL, DB_USER, DB_PASSWORD`
- Option : Docker Compose pour backend + DB
- Déploiement isolé et reproductible